

# 第 10 章：Agentic Systems、Coding Agent 与长期任务

## 10.1 本章符号说明

| 符号/缩写             | English                           | 中文       | 含义                                        |
|-------------------|-----------------------------------|----------|-------------------------------------------|
| Agentic AI        | Agentic Artificial Intelligence   | 智能体式 AI  | 能持续规划、执行、观察和修正的 AI 系统。                    |
| Coding Agent      | Coding Agent                      | 编程智能体    | 能读仓库、改代码、跑测试、修 bug 的 agent。               |
| Computer Use      | Computer Use                      | 计算机使用    | 模型操作 GUI、浏览器、文件和桌面应用的能力。                  |
| GUI               | Graphical User Interface          | 图形用户界面   | agent 可以点击、输入、观察的可视化界面。                   |
| Browser Agent     | Browser Agent                     | 浏览器智能体   | 能搜索、点击、阅读网页并完成网页任务的 agent。                |
| Long-horizon Task | Long-horizon Task                 | 长周期任务    | 需要很多步骤、较长时间和状态保持的任务。                      |
| API               | Application Programming Interface | 应用程序编程接口 | agent 调用外部业务系统或工具的接口。                     |
| Tool Use          | Tool Use                          | 工具调用     | 使用 shell、browser、editor、database、API 等工具。 |
| Tool Error        | Tool Error                        | 工具错误     | 工具调用参数错、执行失败、状态不一致等。                      |
| Agent Harness     | Agent Harness                     | 智能体执行框架  | 管理上下文、工具、权限、状态、trace、恢复和验证的系统层。           |
| Scaffold          | Scaffold                          | 外部脚手架    | agent 外层的控制循环、工具、记忆和验证代码。                 |
| Planner           | Planner                           | 规划器      | 拆任务和安排顺序的模块。                              |
| Executor          | Executor                          | 执行器      | 执行工具和产生修改的模块。                             |
| Critic            | Critic                            | 评审器      | 审查计划、代码、答案和安全风险的模块。                       |

| 符号/缩写          | English                          | 中文            | 含义                     |
|----------------|----------------------------------|---------------|------------------------|
| Verifier       | Verifier                         | 验证器           | 运行测试、检查状态或引用的模块。       |
| SWE-bench      | Software Engineering Benchmark   | 软件工程基准        | 真实 GitHub issue 修复任务。  |
| Terminal-Bench | Terminal Benchmark               | 终端任务基准        | 命令行环境长期任务。             |
| OSWorld        | Operating System World Benchmark | 操作系统世界基准      | GUI/computer-use 任务评测。 |
| TAU-bench      | Tool-Agent-User Benchmark        | 工具-智能体-用户交互基准 | 业务 API 多轮交互任务。         |
| BrowseComp     | Browsing Competition Benchmark   | 浏览检索基准        | 难查信息的网页浏览任务。           |

## 10.2 本章目标

- 理解 agentic system 的核心结构。
- 能讲清 coding agent 和普通代码补全的差别。
- 能设计一个生产可用 agent 的状态、工具、权限、验证和观测方案。
- 能识别 agent eval 的常见陷阱。

## 10.3 Agentic AI 的核心循环

Agent 的最小闭环：

```
goal -> plan -> act -> observe -> verify -> update plan -> finish
```

和普通 LLM API 的差异：

- 普通 LLM 主要是单轮生成。
- Agent 是多轮决策系统。
- Agent 的能力来自模型 + scaffold + tools + verifier。
- Agent 的风险来自写权限、长上下文、外部不可信信息和自动执行。

面试表达：

Agentic AI 不是单个模型能力，而是一个闭环系统。模型负责推理和决策，scaffold 负责状态、工具、预算、权限和验证。生产系统真正难的是可控性和可评估性。

## 10.4 Coding Agent

Coding agent 的典型流程：

- 1. 读 issue / 用户需求。
- 2. 搜索仓库结构。
- 3. 找相关文件和测试。
- 4. 制定修改计划。
- 5. 编辑代码。
- 6. 运行测试或静态检查。
- 7. 根据失败日志修复。
- 8. 输出 diff、验证结果和风险。

和代码补全的区别：

| 能力   | Code Completion | Coding Agent       |
|------|-----------------|--------------------|
| 上下文  | 当前文件附近          | 整个仓库、issue、测试、历史日志 |
| 动作   | 生成片段            | 搜索、编辑、运行、调试、验证     |
| 反馈   | 人类看结果           | 工具和测试实时反馈          |
| 任务长度 | 秒级              | 分钟到小时级             |
| 评价   | 代码片段质量          | 最终 issue 是否解决      |

## 10.5 Long-horizon 为什么难

长期任务常见失败模式：

- 目标漂移：做着做着偏离原始需求。
- 状态遗忘：忘了已经尝试过什么。
- 局部修复：修了当前测试，破坏其他行为。
- 工具错误：命令失败但模型没发现。
- 循环卡住：重复同一动作。
- 过度修改：改太多无关文件。
- 自信汇报：没有验证却声称完成。

解决思路：

- 明确任务状态机。
- 每步记录 trace。
- 对写操作做 diff 和最小权限。
- 测试失败必须回读日志。
- 设置预算、重试上限和终止条件。
- 最终输出必须包含验证证据。

## 10.6 工具和权限设计

Agent 工具不是越多越好。每个工具都要明确：

| 设计项        | English       | 中文   | 说明                                   |
|------------|---------------|------|--------------------------------------|
| Schema     | Tool Schema   | 工具接口 | 参数类型、必填项、约束。                         |
| Permission | Permission    | 权限   | read/write/delete/network/payment 等。 |
| Sandbox    | Sandbox       | 沙箱   | 限制文件系统、网络和命令。                        |
| Approval   | Approval Gate | 审批门  | 高风险动作需要人类确认。                         |
| Timeout    | Timeout       | 超时   | 防止无限执行。                              |
| Rollback   | Rollback      | 回滚   | 出错可恢复。                               |
| Audit Log  | Audit Log     | 审计日志 | 记录动作和结果。                             |

面试回答：

我会把工具按风险分层：只读工具默认允许，写工具需要 diff，破坏性工具需要审批，外部副作用工具需要强约束和审计。

## 10.7 多 Agent 与 Agent Swarm

多 agent 不等于更强，常见模式包括：

- Planner + Executor：一个规划，一个执行。
- Developer + Reviewer：一个写，一个审。
- Researcher + Synthesizer：一个查资料，一个汇总。
- Parallel Attempts：多个 agent 并行尝试，再由 verifier 选择。
- Specialist Agents：不同 agent 负责前端、后端、测试、文档。

风险：

- 通信成本高。
- 错误会互相放大。
- 没有统一状态时容易冲突。
- verifier 不强时，多 agent 只是更贵的幻觉。

面试表达：

多 agent 的价值在于并行探索和角色分工，但必须有共享状态、冲突解决和最终 verifier。否则只是把单 agent 的不稳定放大。

## 10.8 Agentic Coding 评测

常见 benchmark：

| Benchmark          | English                        | 中文         | 关注点                 |
|--------------------|--------------------------------|------------|---------------------|
| SWE-bench          | Software Engineering Benchmark | 软件工程基准     | 真实 GitHub issue 修复。 |
| SWE-bench Verified | SWE-bench Verified             | 人工验证软件工程基准 | 更可靠的真实 issue 子集。    |
| Terminal-Bench     | Terminal Benchmark             | 终端任务基准     | 命令行多步任务。            |
| OSWorld            | Operating System World         | 操作系统世界     | GUI/computer use。   |
| BrowseComp         | Browsing Competition Benchmark | 浏览检索基准     | 多步网页查找。             |
| TAU-bench          | Tool-Agent-User Benchmark      | 工具-用户交互基准  | 业务 API、用户对话和规则遵循。   |

评测时必须问：

- 是否用了同样 scaffold？
- 是否允许浏览器、shell、测试？
- 是否 pass@1 还是多次采样？
- 是否有隐藏测试？
- 是否计算成本和延迟？
- 是否记录 tool error？
- 是否做安全和权限评估？

## 10.9 Agent Harness 为什么重要

同一个 base model 放在不同 agent harness 里，表现可能完全不同。Harness 会决定：

- 模型看到多少上下文。
- 是否有文件搜索、浏览器、shell、测试运行器。
- 工具失败后是否自动恢复。
- 是否允许并行尝试。
- 是否有 reviewer / verifier。
- 写操作是否需要审批。
- 最终 trace 是否可回放。

面试表达：

Agent 能力不是 base model 单独决定的，而是 model + scaffold + harness + tools + budget 的组合。2026 年看 coding agent benchmark 时，必须问清楚 harness 和 protocol，否则分数不可比。

## 10.10 Agent 安全

Agent 比聊天模型风险更高，因为它可以行动。

关键风险：

- Prompt injection：网页或文档指挥模型忽略系统指令。
- Data exfiltration：外部内容诱导模型泄露机密。
- Tool misuse：模型调用不该调用的工具。
- Over-delegation：用户把高风险决策完全交给 agent。
- Reward hacking：为了通过测试修改测试或绕过规则。

安全策略：

1. 工具输出默认不可信。
2. 明确系统指令优先级。
3. 高风险动作 human-in-the-loop。
4. 审计每个 tool call。
5. verifier 检查最终状态，不只检查回答文本。
6. 对 prompt injection 做专门红队数据集。

## 10.11 生产系统 Observability

Agent 上线要监控：

- task success rate。
- tool call count。
- tool error rate。
- retry count。
- latency。
- token cost。
- human escalation rate。
- rollback rate。
- safety violation rate。
- 用户修正率和满意度。

面试表达：

Agent 产品不是 demo 成功就行。要能观察每一步 tool call、失败原因、成本、用户接管点和安全事件，否则出了问题无法 debug。

## 10.12 本章面试高频问题

### 10.12.1 Coding agent 为什么比代码补全难？

因为它要理解仓库、定位 bug、编辑多个文件、运行测试、读失败日志、持续修正，并保证最终状态正确。这是长周期闭环任务，不是单次 token completion。

### 10.12.2 为什么 benchmark 结果经常不可比？

不同模型可能用了不同 scaffold、工具、采样次数、预算、测试可见性和人工干预。agent benchmark 必须报告 evaluation protocol。

### 10.12.3 Agent 怎么防 prompt injection?

把工具输出作为不可信数据，区分系统指令/用户指令/外部内容；敏感工具做权限隔离和审批；最终动作由 verifier 和 policy guard 检查。

### 10.12.4 多 agent 有什么价值?

适合并行探索、角色分工和互审。但只有在有共享状态、冲突解决、强 verifier 时才稳定，否则会增加成本和错误传播。

### 10.12.5 设计一个 coding agent 项目怎么讲?

按“任务-工具-状态-验证-评测-安全”讲。任务是 issue 修复；工具是代码搜索、编辑、shell、测试；状态是计划和尝试记录；验证是单测/回归测试；评测是 SWE-bench 类任务和内部 issue；安全是最小权限、diff 审批和日志审计。

## 10.13 本章 References

---

- [ReAct: Synergizing Reasoning and Acting in Language Models](#)
- [Toolformer: Language Models Can Teach Themselves to Use Tools](#)
- [AgentBench: Evaluating LLMs as Agents](#)
- [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#)
- [TAU-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains](#)
- [BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents](#)
- [Kimi K2: Open Agentic Intelligence](#)
- [Open-World Evaluations for Measuring Frontier AI Capabilities](#)
- [Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows](#)
- [Claude Opus 4.7](#)
- [Introducing GPT-5.5](#)

[章节索引](#)

[← 上一章](#) [下一章 →](#)